
Code-Flow

Design Documents / UML Diagrams

PCCCS495: Advanced Programming (OOP) Lab

Student Name	Ayushman Saha
Roll Number	72
Course	PCCCS495 - Advanced Programming (OOP) Lab
Instructor(s)	Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas
Session	Spring 2026
Institution	IEM / UEM Kolkata

This document contains the design documents and UML diagrams for the Code-Flow desktop text editor project. It is intended for repository submission and instructor review.

Submission Note

The report is organized so that the architecture, class responsibilities, and interaction flow are easy to review without the full source listing.

1. Problem Statement

Standard text editors often hide the internal structure that makes editing reliable and maintainable. Code-Flow addresses this by providing a lightweight Java Swing editor built around clear architecture, reversible edit actions, and responsive file handling. The project focuses on three core goals:

- **Undo/Redo from scratch:** Each edit is modeled as a command object so the application can move backward and forward through history without copying the whole document.
- **Responsive file operations:** Open and save actions are handled with buffered I/O so the interface remains usable during file access.
- **Consistent appearance:** Theme changes are controlled centrally so the editor, menus, status bar, and dialogs stay visually aligned.

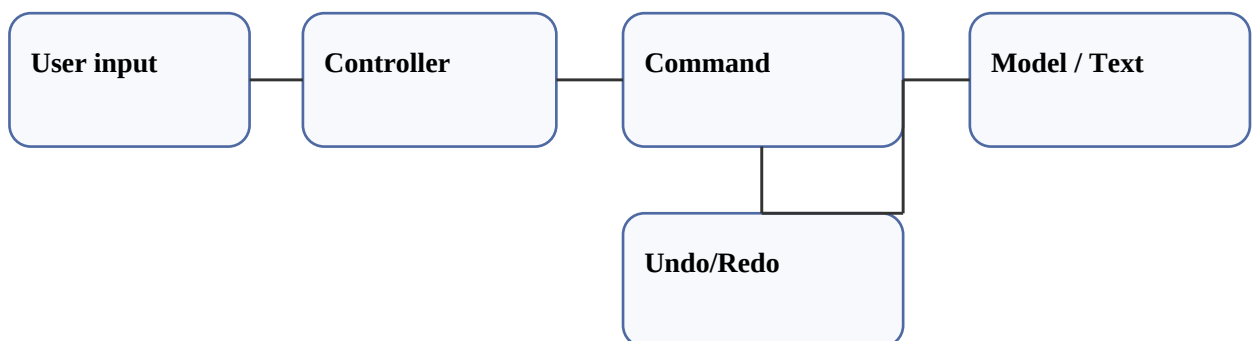
The target users are engineering students and developers who need a distraction-free desktop editor and a project that clearly demonstrates object-oriented design choices.

2. System Design

2.1 High-Level Architecture

The editor follows the MVC idea at a conceptual level. The view presents the Swing user interface, the controller handles events and file actions, and the model represents the editable document and command history.

Interaction Flow



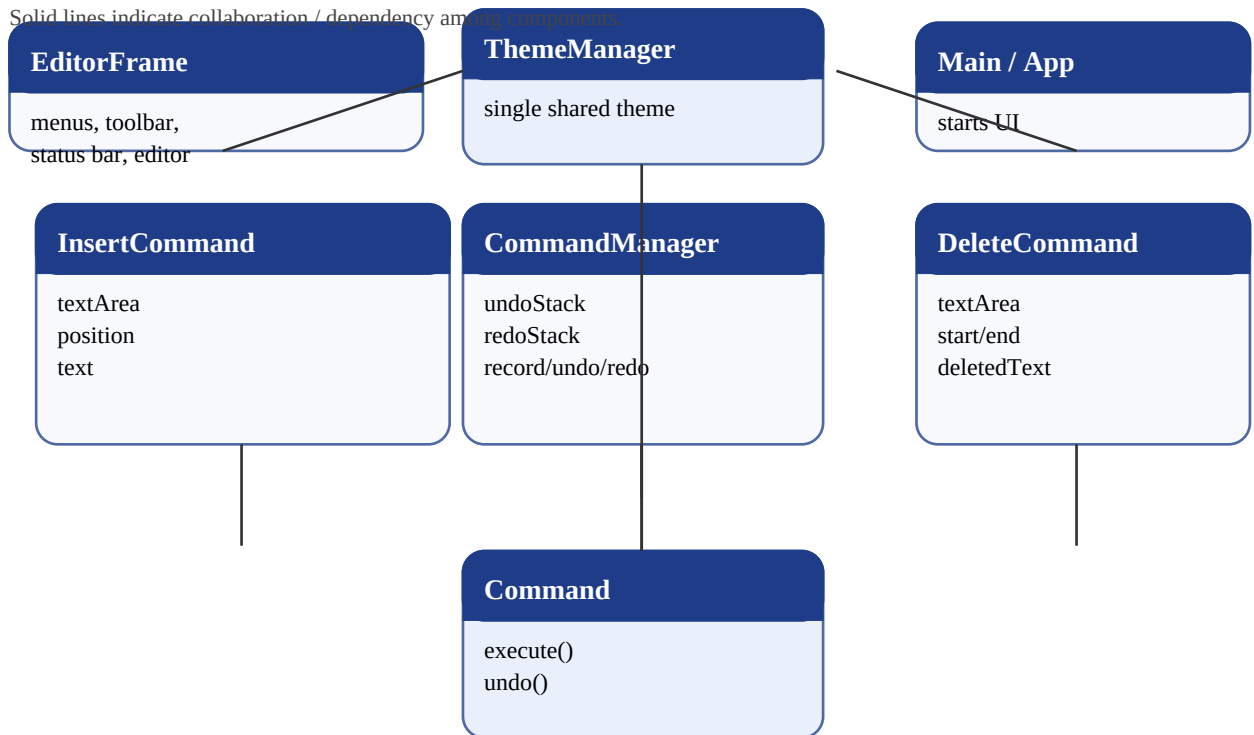
2.2 Package Structure

Package / Component	Responsibility
CodeFlowEditor.java	Application entry point and core editor frame in the final submitted version
Command / InsertCommand / DeleteCommand / CommandManager	Custom command objects for undo and redo operations
Theme logic	Stores and switches between light and dark appearance settings
UI elements	Toolbar, editor area, menu actions, and status display

2.3 UML Class Diagram

UML Class Diagram - Code-Flow editor core

Solid lines indicate collaboration / dependency among components.



3. Major Classes and Responsibilities

Class / Role	Responsibility
Command	Defines the execute() and undo() contract used by all reversible actions
InsertCommand	Stores text inserted by the user and can remove it during undo
DeleteCommand	Stores removed text and can restore it during undo
EditorFrame	Builds the Swing window, menus, toolbar, editor, and status bar
Command stack	Maintains undo and redo history in a controlled order
Theme manager	Applies light and dark styling consistently across the interface

The command objects are intentionally small. They only carry enough state to reverse one action, which keeps the design easy to explain in viva and supports a clean undo/redo story.

4. OOP Concepts Demonstrated

4.1 Abstraction

The Command interface hides the details of how editing actions work. The rest of the editor only asks a command to execute or undo an action.

4.2 Inheritance

The Swing window class extends JFrame so the application can use built-in window behavior while adding custom editor features.

4.3 Polymorphism

Undo and redo work on a stack of Command references. At runtime, the correct action is chosen automatically whether the object is an insert command or a delete command.

4.4 Design Pattern

The project uses the Command pattern for reversible edits and a singleton-style theme manager for consistent UI styling.

5. Technical Highlight

The most important design decision is the custom command-based undo/redo mechanism. Instead of relying on a generic editor history component, every edit is represented as an object. This makes the project easier to justify academically and gives full control over how actions are reversed.

Why this choice	Benefit
Command objects	Better separation between UI actions and state changes
Stack-based history	Natural undo/redo order
Central theme control	Consistent appearance in all UI elements

6. Testing and Error Handling

The project is checked for blank-document startup, save and open behavior, undo and redo boundaries, unsaved changes on exit, and file-access failures. The UI should remain stable even when the user cancels a save dialog or attempts to close the application without saving.

- Opening the application should show an empty editor.
- Saving should preserve the current text exactly.
- Undo and redo should restore earlier text states in order.
- Closing a modified document should show a save/discard prompt.
- Invalid file operations should show a clear error message instead of crashing.

7. Conclusion and Future Scope

Code-Flow demonstrates how a desktop Java Swing application can be organized around strong object-oriented ideas. The design keeps user interaction, reversible commands, and theme control clearly separated, which makes the editor practical for a semester project and easy to discuss in viva.

Future scope

- Search and replace dialog with match highlighting
- Line numbers and syntax highlighting
- Tabbed editing for multiple files
- Recent files and autosave support

Appendix: attach the project proposal PDF and the approval mail screenshot in the repository as supporting submission material.